In SQL, a window function with `OVER()` performs a calculation across a set of related rows while keeping every individual row in the result. Unlike `GROUP BY`, it does not collapse multiple rows into one.

Let's understand it with Oracle SQL examples.

1. Sample employee table

Suppose we have this table:

EMP_ID	NAME	DEPT	SALARY
1	Amit	IT	50000
2	Rahul	IT	70000
3	Priya	IT	60000
4	Neha	HR	40000
5	Ravi	HR	50000

2. `SUM() OVER()` — total salary by department

```
SELECT
    emp_id,
    name,
    dept,
    salary,
    SUM(salary) OVER (
        PARTITION BY dept
    ) AS dept_total_salary
FROM employee;
```

Result:

NAME	DEPT	SALARY	DEPT_TOTAL_SALARY
Amit	IT	50000	180000
Rahul	IT	70000	180000
Priya	IT	60000	180000
Neha	HR	40000	90000
Ravi	HR	50000	90000

How it works:

- `SUM(salary)` calculates the sum.
- `OVER()` makes it a window calculation.
- `PARTITION BY dept` divides rows into department groups.
- Each row remains in the result.

3. `GROUP BY` versus `OVER()`

With `GROUP BY`:

```
SELECT dept, SUM(salary) AS total_salary
FROM employee
GROUP BY dept;
```

Result: only two rows, one per department.

With `OVER()`:

```
SELECT
   name,
```

```
    dept,
    salary,
    SUM(salary) OVER (PARTITION BY dept) AS total_salary
FROM employee;
```

Result: all five employees remain, with their department total alongside each employee.

4. `ORDER BY` inside `OVER()` — running total

```
SELECT
    name,
    dept,
    salary,
    SUM(salary) OVER (
        PARTITION BY dept
        ORDER BY salary
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS running_total
FROM employee;
```

For IT, the running total follows ascending salary:

NAME	SALARY	RUNNING_TOTAL
Amit	50000	50000
Priya	60000	110000
Rahul	70000	180000

The window frame means: start at the first row in the department and accumulate through the current row.

5. Ranking functions

Window functions are particularly useful for ranking employees within departments.

`ROW_NUMBER()`

Assigns a unique sequential number to each row.

```
SELECT
   name,
   dept,
   salary,
   ROW_NUMBER() OVER (
      PARTITION BY dept
      ORDER BY salary DESC
   ) AS rn
FROM employee;
```

`RANK()` versus `DENSE_RANK()`

Suppose salaries are `90000, 80000, 80000, 70000`.

SALARY	ROW_NUMBER	RANK	DENSE_RANK
90000	1	1	1
80000	2	2	2
80000	3	2	2
70000	4	4	3

- `ROW_NUMBER()` gives each row a unique number.
- `RANK()` leaves gaps after ties.
- `DENSE_RANK()` does not leave gaps.

6. Common window functions

Function	Purpose
`SUM() OVER()`	Total or running sum
`AVG() OVER()`	Average by window
`COUNT() OVER()`	Count rows in window
`ROW_NUMBER()`	Unique row numbering
`RANK()`	Ranking with gaps
`DENSE_RANK()`	Ranking without gaps
`LAG()`	Access a previous row's value
`LEAD()`	Access a following row's value
`FIRST_VALUE()`	First value in a window
`LAST_VALUE()`	Last value in a window, subject to the frame

7. The three important parts of `OVER()`

```
FUNCTION(column) OVER (
   PARTITION BY column
   ORDER BY column
   ROWS BETWEEN ... AND ...
)
```

- `PARTITION BY` — divides rows into independent groups.
- `ORDER BY` — establishes the logical order within each group.
- Window frame — specifies which rows around the current row participate in the calculation.

You can omit any of these clauses when they aren't needed.

Interview tip: Practise these three problems first:

1. Find the second-highest salary in each department using `DENSE_RANK()`.
2. Calculate a running total using `SUM() OVER (ORDER BY ...)`.
3. Compare each employee's salary with the previous employee using `LAG()`.

These cover many of the most common SQL window-function interview questions.